EduGenie: Google Gemini-Powered AI Learning Assistant

Project Description:

EduGenie harnesses the power of Generative AI to provide a highly personalized, intelligent academic assistance framework, offering users an intuitive and multi-engine educational workspace. The platform includes five core pillars: a Q&A Engine for precise context-driven extraction, a Concept Explainer that deconstructs complex topics into specific cognitive target depths using analogies, a dynamic Quiz Generator that curates interactive diagnostic tests to gauge comprehension, a high-density Text Summarizer for rapid information compression, and a Personalized Learning Path architect that builds structural timeline milestones for target educational goals.

Utilizing Google's advanced API with the Gemini 2.5 Flash model alongside a lightweight local CPU-fallback pipeline featuring LaMini-Flan-T5, EduGenie processes complex educational materials seamlessly to deliver instant, high-fidelity responses. Built natively with an asynchronous FastAPI backend layer and an interactive Jinja2 HTML/CSS frontend template, the application ensures real-time DOM injection without requiring page refreshes. Backed by isolated component logic and robust Docker containerization, EduGenie empowers students, educators, and lifelong learners to scale their technical proficiency and master new disciplines with absolute confidence.

Scenarios

Scenario 1:

A student pastes a long, technical textbook chapter into the Q&A Engine workspace and asks for a direct explanation of a complex formula. The system immediately parses the context block and generates a precise answer using academic standards, alongside specific page inferences, in seconds.

Scenario 2:

An educator needs a quick diagnostic check to evaluate classroom comprehension on a topic. They upload the reading material to the Quiz Generator, and the platform dynamically structures a multiple-choice interactive test with hidden answer key matching, enabling instant automated student grading on the page.

Scenario 3:

A professional seeking a career transition wants to master full-stack software development in a limited window. They input their ultimate objective and timeline constraints into the Learning Path module, and the AI maps out a chronological, milestone-driven curriculum timeline complete with objective progress milestones.

Technical Architecture:

Description:

EduGenie utilizes a modular, decoupled three-tier architecture to deliver rapid, asynchronous AI-driven educational features. The frontend provides a fully responsive student workspace interface built using HTML5, CSS3, and Vanilla JavaScript, handled dynamically via Jinja2 Templates.

The backend relies on the high-performance FastAPI web framework, orchestrating clean data validation schemas using Pydantic models along with robust runtime exception handling. It interfaces directly with cloud-hosted infrastructure via the Google GenAI SDK using the Gemini 2.5 Flash model for complex text comprehension tasks, while maintaining a local CPU-optimized machine learning pipeline with LaMini-Flan-T5 for fast quiz fallback operations. The application is completely containerized using a custom Dockerfile and deployed publicly on the web through Hugging Face Spaces.

Pre-requisites:

FastAPI Framework Knowledge: FastAPI Documentation

Google GenAI SDK Integration: Google AI Studio Gemini API Reference

HTML, CSS, and JavaScript

Skills: W3Schools HTML/CSS/JS Web Development Tutorials

Python Programming Proficiency: Python Language Documentation

Version Control with Git: Git Documentation Manual

Development Environment Setup: FastAPI Local Deployment Guide

Containerization & Cloud Hosting: Docker Engines & Hugging Face Spaces Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Obtain a Google Gemini API key from Google AI Studio and configure it securely inside the system environment parameters.
- **Activity 1.2:** Research and evaluate target Generative AI platforms, selecting Gemini 2.5 Flash for deep cognitive tasks and LaMini-Flan-T5 as a lightweight local fallback pipeline.
- **Activity 1.3:** Define the modular application architecture, mapping out isolated file blocks and data streams between the frontend workspace, FastAPI backend router, and the model inference layers.
- **Activity 1.4:** Establish the local development environment, initializing a virtual environment and configuring system package dependencies via the requirements.txt manifest.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Build independent, task-specific Python script files for the five core educational pillars: qna.py, explanation_module.py, quiz_module.py, summary_module.py, and learning_path.py.
- **Activity 2.2:** Integrate advanced prompt engineering techniques, such as forcing structured outputs using specific payload separator keywords (|||ANSWERKEY|||) for programmatic parsing.

Milestone 3: App.py Development

- **Activity 3.1:** Write the central application framework logic inside main.py, constructing secure POST endpoints validated by data-binding Pydantic schema models.

Milestone 4: Frontend Development

- **Activity 4.1:** Structure a clean, fully responsive single-page web dashboard using HTML5 and CSS3, including functional loading spinner keyframe animations.
- **Activity 4.2:** Embed advanced asynchronous client-side JavaScript loops to capture workspace form inputs, process API fetch requests, and implement dynamic real-time DOM updates.

Milestone 5: Deployment

- **Activity 5.1:** Execute comprehensive local debugging test passes across all tabs and API routes using the Uvicorn ASGI server utility.
- **Activity 5.2:** Draft a production-grade custom application environment config via a Dockerfile.
- **Activity 5.3:** Deploy the fully containerized EduGenie workspace to a public Hugging Face Spaces platform runtime environment.

Milestone 6: Conclusion

The development of the EduGenie AI Learning Assistant successfully demonstrates how modern web frameworks like FastAPI can be combined with Large Language Models to create an interactive, responsive educational platform. By utilizing Gemini 2.5 Flash for advanced reasoning and a local LaMini-Flan-T5 model for fallback tasks, the system provides reliable, real-time educational toolsets.

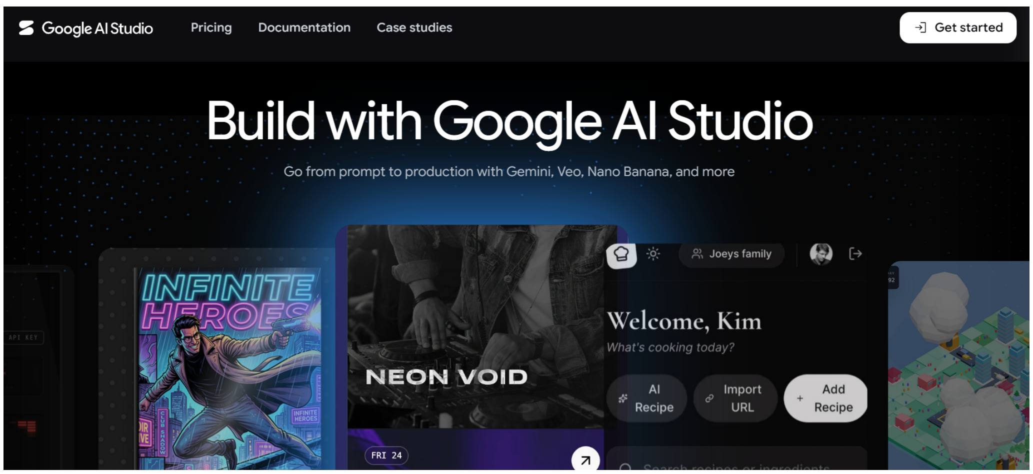
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI models from Google for our multi-functional educational assistant needs. This involves researching the capabilities and performance of various models that Google AI Studio offers, ensuring that the chosen models align well with the application's objectives of generating high-fidelity explanations, Q&A responses, summaries, interactive quizzes, and structured learning pathways.

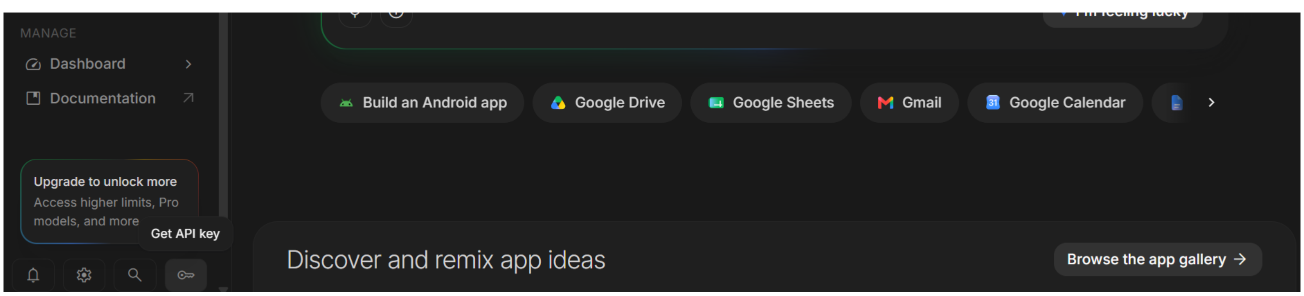
Activity 1.1: Generate a Google Gemini API Key

Before the application can communicate with the Gemini LLM ecosystem, you need a valid Google Gemini API key. Follow the steps below to create one and connect it securely to the project:

- **Step 1 — Create a Google AI Studio Account:** Visit the official Google AI Studio dashboard at google.com and sign up for a free developer account using your Google credentials.

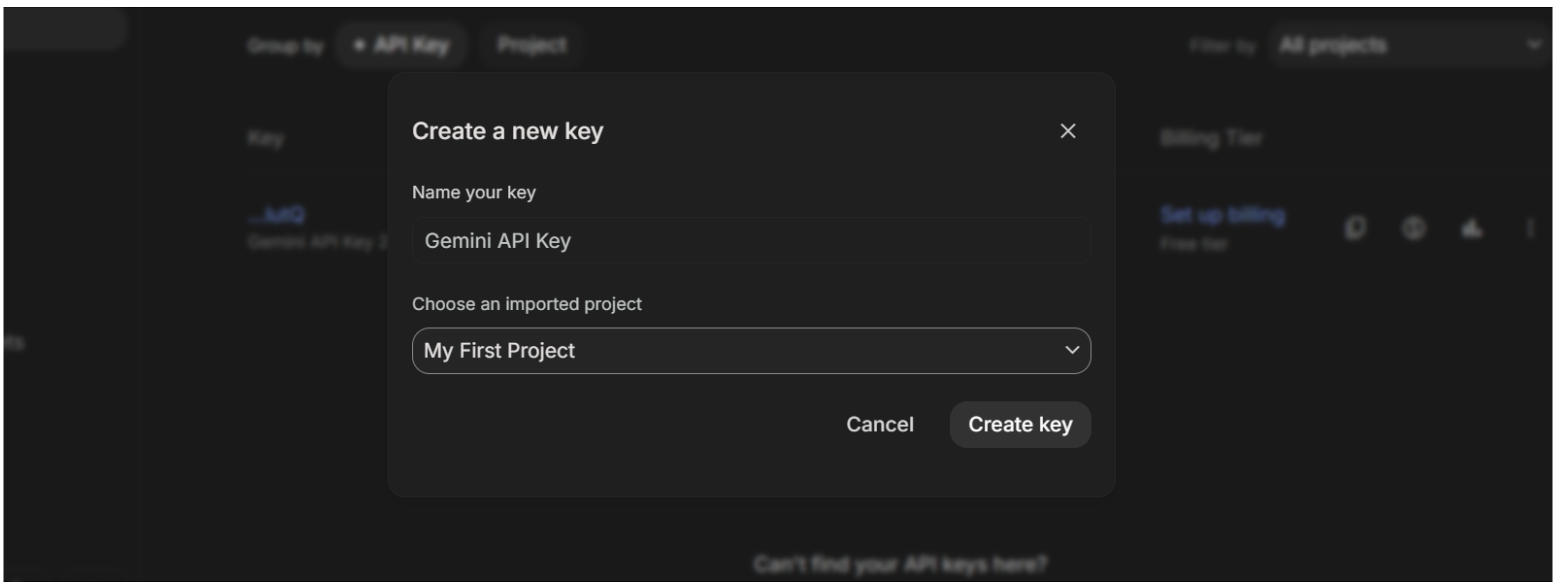


- **Step 2 — Navigate to API Keys :** After logging in, locate and click on the "Get API Key" button positioned in the top-left or main dashboard navigation area to enter the key management panel.



- **Step 3 — Create new API Key :** Click on "Create API Key", choose your active cloud project environment, and copy the newly generated secret string token.

Step 4 - API Key : A dialog prompt will generate your unique access string. Copy the string immediately and save it in a secure location; for security reasons, Google AI Studio hides the full sensitive value once you close the window.



- **Step 5 — Add the Key to Your Project:** Expose the secret credential variable directly to your active server terminal environment framework using the command matching your operating system shell instance:

  Windows CMD: set GEMINI_API_KEY=your_copied_api_key_here
  Windows PowerShell: $env:GEMINI_API_KEY="your_copied_api_key_here"
  Mac/Linux: export GEMINI_API_KEY="your_copied_api_key_here"

- **Step 6 — Verify the Key is Loaded:** The core backend application main.py fetches this environment token automatically upon starting up using standard os library wrappers:

```
import os
api_key = os.getenv("GEMINI_API_KEY")
```

Important : Never hardcode your active API tokens directly into your script files when pushing to public repositories. When deploying live on Hugging Face Spaces, configure this key inside the "Repository Secrets" settings dashboard layer to hide your environment credentials completely.

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the critical research evaluation steps required to select the optimal model setup for the project:

- Understand the Project Requirements: Review the multi-functional needs of the EduGenie application, focusing on the specific academic workflows it handles (Concept Explanation, context-based QnA, structural Quiz generation, Text Summarization, and Personalized Learning Paths).
- Explore Google and Hugging Face Model Ecosystems: Visit the Google AI Studio documentation to analyze available LLM architectures alongside local Hugging Face model families to assess contextwindow sizes, processing speeds, reasoning strengths, and parameter costs.
- Evaluate Model Performance: Compare language models based on response speed, structural reasoning quality, accuracy in extracting reference material, and applicability to varied educational domains.
- Select the Optimal Models: Choose Gemini 2.5 Flash for its expansive context capabilities, rapid processing times, and advanced cognitive accuracy across reasoning tasks. Pair it with LaMini-Flan-T5 as a lightweight local model to run fast fallback quiz tasks locally on memory-restricted hardware.

Activity 1.3: Define the Architecture of the Application

- Draft an Architectural Diagram: Sketch a three-tier system layout mapping the core application building blocks, highlighting structural interactions between the browser UI layer (HTML, CSS, JS), the FastAPI router controller, and the dual AI processing engines.
- Detail Frontend Functionality: Outline how users engage with the dashboard workspace, inputting reference textbooks or goals, choosing specific tasks from tab bars, selecting specialized parameters (like explanation depth or quiz count), and reviewing dynamic real-time outputs without page refreshes.
- Outline Backend Responsibilities: Specify how the FastAPI application receives asynchronous incoming client payloads, uses Pydantic schemas to validate parameters on the fly, manages modular routing files, and safely catches server exceptions.
- Describe AI Integration Points: Define how the system constructs complex prompts inside individual feature \modules (qna.py, quiz_module.py, etc.), passes the data stream via the google-genai SDK interface, handles hidden output layout separators like |||ANSWERKEY||| for quizzes, and returns formatted strings to the user.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.10+ is installed on your machine along with pip for project dependency management.
- **Create a Virtual Environment:** Set up an isolated virtual environment using venv to avoid package conflicts:

```
D:\EduGenie> python -m venv venv
D:\EduGenie> venv\Scripts\activate
```

```
(venv) D:\EduGenie> pip install -r requirements.txt
```

- **Install Fast API Framework:** Use pip to install

```
(venv) D:\EduGenie> pip install fastapi uvicorn jinja2
```

- **Install Google GenAi SDK:** Install the official client libraries required to communicate directly

```
(venv) D:\EduGenie> pip install google-genai
```

- **Install Local NLP:** Setup the AI/ML frameworks required to download

```
(venv) D:\EduGenie> pip install transformers torch
```

- **Configure the API Key:** Set your secure authorization token directly as an environment variable in the active terminal window instance:

```
(venv) D:\EduGenie> set GEMINI_API_KEY=your_actual_api_key_here
```

- **Set Up Application Structure:** Create the root directory architecture including template engines, static folder paths for layouts, modular Python scripts, and main files as mapped below:

```
EDUGENIE/
│
├── static/
│   └── style.css
│
├── templates/
│   └── index.html
│
├── explanation_module.py
├── learning_path.py
├── main.py
├── qna.py
├── quiz_module.py
├── requirements.txt
├── README.md
└── summary_module.py
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the five isolated core educational functionalities driven by highly targeted systemic prompt engineering:

- Q&A Engine (qna.py): Parses user-provided textbook chapters or notes and answers questions precisely based on that context. It falls back to internal academic knowledge if information is missing from the source text.
- Concept Explainer (explanation_module.py): Breaks down complex topics (e.g., Quantum Entanglement) into digestible explanations tailored to a selected cognitive target depth (e.g., "5-Year-Old Level" with heavy analogies vs. "Rigorous Technical Breakdown").
- Quiz Generator (quiz_module.py): Dynamically structures multiple-choice evaluation tests from input texts. It utilizes a custom prompt divider (|||ANSWERKEY|||) to split questions from correct answers for programmatic parsing and grading.
- Text Summarizer (summary_module.py): Executes high-density content compression based on user preferences, rendering either structural bullet matrices or abstract summary cards.
- Learning Paths Architect (learning_path.py): Maps out chronological learning milestone plans, organizing clear, weekly educational progression benchmarks to achieve a student's study goals.

Activity 2.2: Implement the Flask Backend

Define Routes in FastAPI:

- Set up asynchronous routing handlers inside main.py to manage the root landing page and the 5 independent API POST workspace targets (/api/qna, /api/explain, /api/quiz, /api/summary, /api/learning-path).

Process User Input:

- Render independent input forms dynamically grouped via a responsive tab-link navigation dashboard inside index.html.
- Capture frontend payloads natively and utilize structured Pydantic models (QnARequest, QuizRequest, etc.) to automatically parse and validate the incoming raw JSON objects.

Integrate API Calls:

- Pass validated string arguments to the respective background python modules where system prompts are compiled dynamically.
- Initialize standard genai.Client instances to transmit active text strings over secure network channels via the modern client.models.generate_content() method using the Gemini 2.5 Flash model model suite.
- Incorporate client-side JavaScript formatting parsers to capture split strings, securely populate background answer arrays, and dynamically update interactive DOM answer select modules on the fly.

Milestone 3: Main.py Development

Milestone 3 focuses on creating and refining the core logic of the main.py file, which serves as the central orchestration backbone of the EduGenie application. In this milestone, you will set up each feature's asynchronous endpoint routing architecture, establish reliable Pydantic model configurations for structured data validation, connect external AI modules, and ensure smooth data payload translation between the client UI and the processing engines.

Activity 3.1: Writing the Main Application Logic in main.py

Define the Core Routes in main.py:

- Establish functional routes for EduGenie's workspace tabs, mapping explicit endpoints for distinct purposes:
  - **/** — Home page route that mounts static resources and renders the main index.html interface template file.
  - **/generate**— POST endpoint that accepts incoming JSON payloads, triggers the interactive test generator pipeline, and returns the dual-parsed structural text output blocks seamlessly.

```
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel
import os
import quiz_module

app = FastAPI(title="EduGenie Learning Assistant")
templates = Jinja2Templates(directory="templates")

# Explicit Pydantic data schemas for data validation
class QuizRequest(BaseModel):
    context: str
    num_questions: int

@app.get("/", response_class=HTMLResponse)
async def serve_workspace(request: Request):
    return templates.TemplateResponse(request=request, name="index.html")

@app.post("/api/quiz")
async def handle_quiz(data: QuizRequest):
    try:
        # Triggers isolated modular logic using input constraints
        result = quiz_module.generate_quiz(data.context, data.num_questions)
        return {"output": result}
    except Exception as err:
        return {"output": f"Server processing error: {str(err)}"}
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for EduGenie. This involves designing a cohesive dark-themed workspace layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering paths using vanilla JavaScript to display AI-generated explanations, Q&A responses, summaries, interactive tests, and milestone learning roadmaps based on real-time user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as a responsive single-page interface (SPA), including a workspace application header with the EduGenie brand logo, a multi-tab modular navigation container, an active form workspace input zone, and dedicated dynamic result section boxes.
- Use semantic HTML elements (like <header>, <main>, <nav>, <section>) to keep the source layout strictly organized and ensure the interface accessibility parameters map perfectly for all users.
- Integrate standardized modern system typography rules inside your header imports to guarantee crisp text presentation and professional scannability across code blocks.

Design a Responsive Layout Using CSS:

- Create a clean, centralized style architecture (static/style.css) implementing a modern deep-slate backdrop aesthetic with visual anchor keyframe transitions to provide rich interface depth.
- Use Flexbox and block element behaviors to organize layout components efficiently, allowing the form parameters to scale uniformly relative to output block margins.
- Incorporate media queries into the stylesheet to ensure your multi-tab horizontal layout dynamically collapses and adapts smoothly across desktop, tablet, and mobile device screen environments.

Create Separate UI Components for Each Output Type:

- Q&A Conversational Containers: Unique text blocks that display extracted answers or reference summaries within structured code-font environments for clear academic parsing.
- Interactive Test Generation Zone: Dynamically generated element matrices featuring customized dropdown option menus (user-ans-i) for multi-choice response collection.
- Learning Milestone Blueprints: Structured chronological text progression blocks formatted with bold phase headers to separate target milestones cleanly.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach modular addEventListener('submit') submission event listeners to forms, automatically executing event.preventDefault() to block standard page reloads and pipe data as JSON string payloads directly to the FastAPI endpoints (/api/qna, /api/quiz, etc.) via the Fetch API.
- Trigger automated UI visibility state changes immediately upon user submission, replacing placeholder values inside destination containers with active instructional text strings and removing the .hidden class from custom CSS spinners to alert the user of active backend model pipelines.

Render Results Dynamically:

- Implement localized programmatic text logic inside the main script container block, dividing complex multi-part AI strings at strict formatting boundaries (|||ANSWERKEY|||) to parse out background answer arrays while formatting user-facing assessment text.
- Iterate structurally through active question lengths to inject corresponding form elements on the fly, transforming raw model parameters into a functional, live text evaluation application inside the local browser DOM environment.

Restore UI State After Generation:

- Utilize the concluding finally runtime loop execution block to hide the pulsing loading wheel animation elements automatically regardless of an HTTP 200 success code or a validation failure state, keeping the application workspace responsive for future user input.

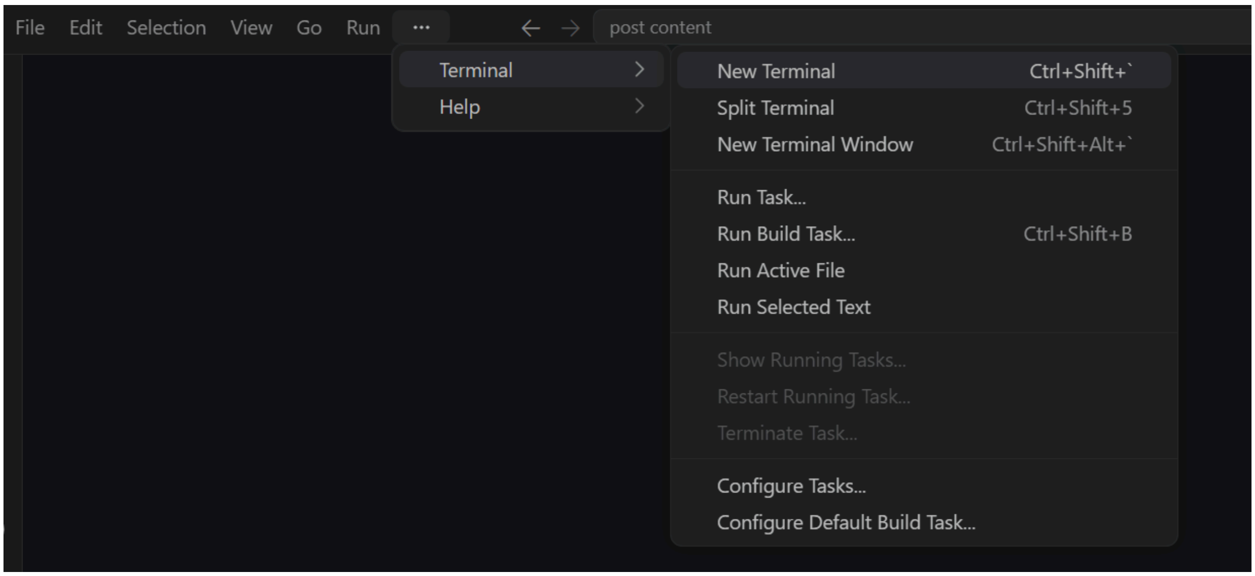
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the EduGenie application first locally to verify correct operation, and then publicly via Hugging Face Spaces to share the app over a secure, cloud-accessible URL. This involves configuring the asynchronous server environment, managing system-level machine learning package dependencies, loading secure authentication keys, and launching the application container runtime for real-world scaling.

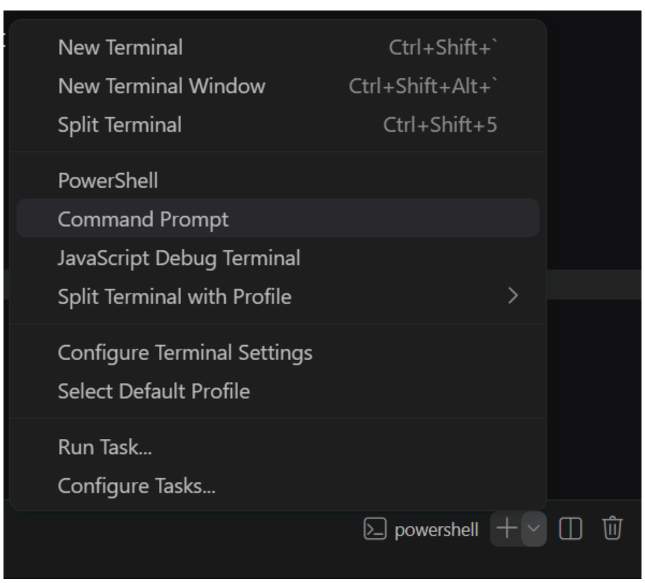
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open "Command Prompt"



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\EduGenie> python -m venv venv
D:\EduGenie> "d:\EduGenie\venv\Scripts\activate"
(venv) D:\EduGenie> pip install -r requirements.txt
```

Configure Environment Variables:

- Windows CMD: set GEMINI_API_KEY=your_actual_api_key_here

Activity 5.2: Local Testing and Verification

Start the Uvicorn Asynchronous Development Server:

- Execute the Local Server Invocations:

```
(venv) D:\EduGenie> uvicorn main:app --reload
```

- Once running, open a browser and navigate to “http://127.0.0.1:8000” to access the app.

```
(venv) D:\EduGenie> uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['D:\\EduGenie']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [12404] using StatReload
INFO:     Started server process [23440]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Activity 5.3: Public Deployment via Hugging Face Spaces

Hugging Face Spaces provides a managed, isolated server runtime to host containerized machine learning applications. Using a custom Docker configuration ensures that the FastAPI application, the remote Gemini SDK, and the local CPU-optimized NLP models run within a production-grade cloud sandbox accessible from a persistent, public URL.

Prepare the Docker Container Configuration (Dockerfile):

- Create a text file named exactly Dockerfile in your project root to specify the application image environment instructions:

```
FROM python:3.10-slim
WORKDIR /code
COPY ./requirements.txt /code/requirements.txt
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "7860"]
```

Initialize the Hugging Face Space Repository:

1. Log into huggingface.co and click "New Space".

2, Name your space (e.g., EduGenie), select Docker as the SDK environment choice, and pick the Blank template workspace configuration.

Configure Secure Repository Secret Variables:

1. To keep your sensitive Google tokens private while pushing code publicly, navigate to your Space's Settings panel tab.

2. Locate the "Variables and secrets" module layer and create a new repository secret:

   Name: GEMINI_API_KEY
   Value: [Your actual Google AI Studio Secret API string key]

Commit and Build the Application Image:

1. Initialize your repository trackers or use the drag-and-drop web UI layout upload system to push all your codebase files (main.py, Dockerfile, templates/, static/, etc.) into your Hugging Face Space file repository.

2. Upon detecting your Dockerfile, the built-in system hooks automatically trigger an isolated image build matrix process. The platform console logs will track dependency downloads and verify server readiness.

```
==> Building container image...
==> Running container validation checks...
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0 (Press CTRL+C to quit)
==> Status: RUNNING. Your public app is live!
```

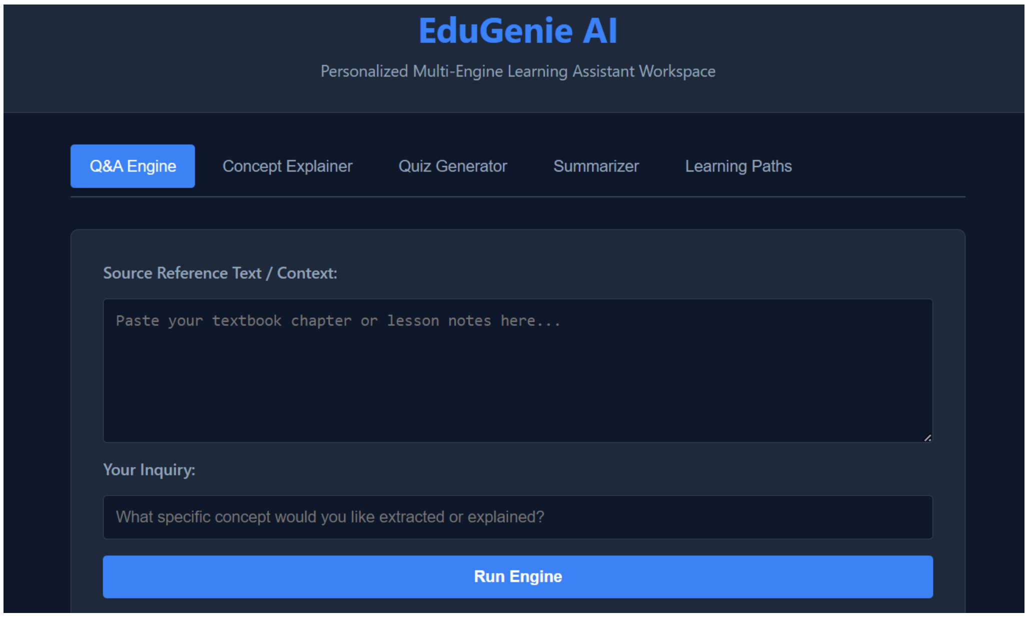
Access and Share the Public Live URL:

1. Once the deployment state shifts to a pulsing green Running badge indicator, the interactive UI workspace becomes immediately operational.

2. Copy the permanent deployment public sandbox address directly from the browser window header to distribute your cloud hosted learning dashboard globally :
URL: https://huggingface.co

Exploring the Website's Web Pages:

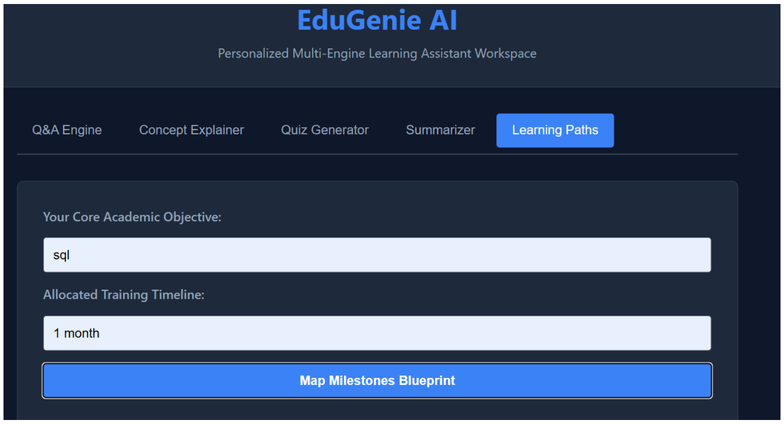
Home / Generator Page:



Description: The single-page workspace interface of EduGenie serves as both the central hub and the active execution dashboard for all educational modules. It features a responsive dark-themed slate grid design layout, a bold application header showcasing the EduGenie brand, and a concise functional subtitle.

The primary layout block features a clean, horizontal multi-tab task switcher list container row that enables students to hop instantly between the core features. The active panel includes input forms, model parameter toggles, task configuration switches, and dynamic target result panels organized cleanly within a single viewport workspace layout.

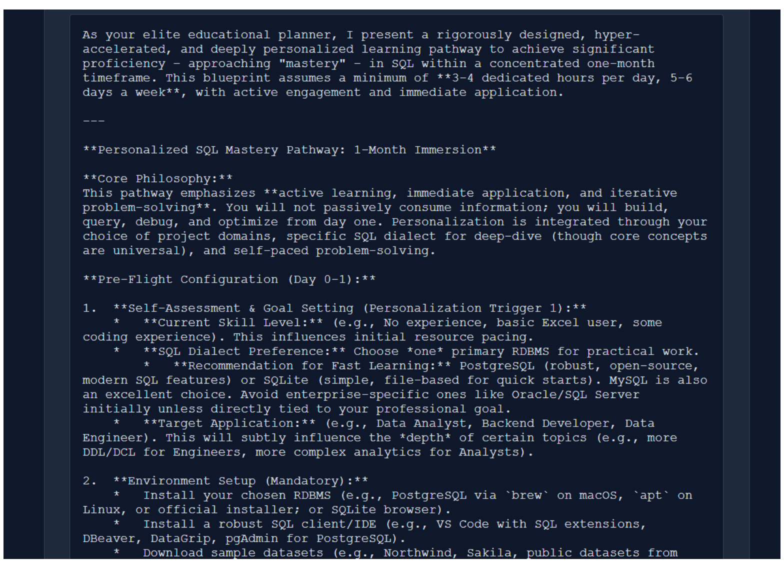
Input Section:



Description:

A styled component block contains the active operational task workspace cards on the page dashboard view grid. Users paste their target source materials, textbook note selections, or complex articles straight into the dedicated context text area element containers.

Results Section:



Description:

The result section of EduGenie is a dynamic output container that sits cleanly beneath the active input workspace forms. To ensure a fast, fluid user experience, this section uses client-side JavaScript to overwrite container elements in real-time, removing the need for slow browser page refreshes.

Conclusion:

EduGenie is an AI-powered personalized learning assistant built with the asynchronous FastAPI framework and styled with a sleek dark-themed workspace UI that modernizes online student engagement. It utilizes Google's high-speed Gemini 2.5 Flash inference API along with a local CPU-optimized LaMini-Flan-T5 model fallback pipeline to instantly deliver tailored context-based Q&A extraction, tiered concept explanations, text summarization bullet matrices, chronological learning path timelines, and fully interactive, auto-graded diagnostic quizzes. The project lifecycle—which spans model orchestration, asynchronous RESTful backend module development, responsive front-end DOM manipulation, and public containerized cloud deployment via a custom Dockerfile on Hugging Face Spaces—highlights how targeted prompt engineering and a decoupled architecture can significantly boost productivity and elevate comprehension retention for students and educators alike.

Looking ahead, EduGenie offers extensive opportunities for future expansion and structural feature growth. Key scalability paths include integrating native markdown rendering libraries to elevate text layout formatting, implementing local SQLite or PostgreSQL database systems to persistently track and save a student's personal study records and test attempt histories, and incorporating secure OAuth2 user session authentication. Furthermore, adding multi-modal capabilities like OCR image processing would allow learners to directly upload snapshots

of physical textbooks or diagrams. By continuously expanding these interface mechanics and refining its structural model prompts, the platform is well-positioned to evolve from a specialized educational tool into a comprehensive, adaptive, full-scale learning management ecosystem.